# Buttons

### 1. Was ist ein Button?

Ein Button löst eine Aktion aus. Er kann ein Formular absenden, Daten zurücksetzen, einen Dialog öffnen oder JavaScript ausführen. Die Wirkung hängt vom HTML und der Anwendung ab; ein erfolgreicher CLICK beweist noch nicht, dass die fachliche Aktion erfolgreich war.

#### Die wichtigen HTML-Varianten

| HTML | Beschriftung | Standardverhalten |
|---|---|---|
| button type=button | Inhalt zwischen den Tags | Keine automatische Formularaktion; häufig JavaScript. |
| button type=submit | Inhalt zwischen den Tags | Formular absenden. |
| button type=reset | Inhalt zwischen den Tags | Formularwerte auf Ausgangswerte zurücksetzen. |
| input type=button | value | Keine automatische Formularaktion. |
| input type=submit | value | Formular absenden. |
| input type=reset | value | Formularwerte zurücksetzen. |

```
<button id="save" type="button">Speichern</button>
<input id="submit" type="submit" value="Absenden">
<button id="reset" type="reset">Zurücksetzen</button>
```

Bei einem Button in einem gewöhnlichen Formular kann ein fehlendes type zur Submit-Wirkung führen. Lernfixtures sollten deshalb type ausdrücklich angeben. Ein input-Element besitzt keinen Inhalt zwischen öffnendem und schließendem Tag.

Ein Element mit `role="button"` kann ebenfalls einen zugänglichen Buttonnamen besitzen. Die Anwendung muss aber Fokus, Tastaturbedienung und Verhalten korrekt implementieren; die Rolle allein macht ein div nicht zu einem funktionsfähigen nativen Button.

### 2. Welchen Namen verwendet button.text?

```
button:
  text: Speichern
```

```
3   exact: true
```

AGATE erzeugt einen Locator nach Rolle BUTTON und zugänglichem Namen. Der Parameter text bezeichnet deshalb nicht immer den reinen DOM-Text:

```
1 <button type="button"><span>Speichern</span></button>
2 <input type="submit" value="Speichern">
3 <button type="button" aria-label="Dokument
  speichern">Speichern</button>
```

Die ersten beiden Elemente haben normalerweise den Namen Speichern. Der dritte wird über Dokument speichern ausgewählt. Bei einem Button mit Icon kann aria-label oder eine Textalternative nötig sein. Dekorative Elemente mit aria-hidden=true sollen den Namen nicht verändern.

#### Span, Formatierung und Leerzeichen

```
1 <button type="button"><span>Jetzt</span> <b>speichern</b></button>
2 <button type="button">  Speichern  </button>
3 <input type="button" value="(O)K ">
```

Verschachtelte Textteile können zum Namen beitragen. Der erste Name ist typischerweise Jetzt speichern. Äußere Leerzeichen und NBSP werden bei der Namensauswahl normalisiert. Der letzte Button kann deshalb mit `(O)K` und exact true gefunden werden, obwohl sein value am Ende ein Leerzeichen enthält.

Ein CSS-Attributvergleich wie `input[value="(O)K"]` erhält diese Normalisierung nicht und passt nicht zu value `(O)K `. Bei Zero-Width-Zeichen nicht pauschal von Normalisierung ausgehen. Mehrere Textteile ohne trennendes Leerzeichen nicht automatisch als getrennte Wörter interpretieren.

#### Accessible Name, Elementtext und value sind verschiedene Dinge

| Beispiel | Locator | Sinnvolle Inhaltsprüfung |
|---|---|---|
| button mit Inhalt Speichern | button.text Speichern | TEXT Speichern |
| input submit mit value Absenden | button.text Absenden | VALUE Absenden |
| button mit aria-label Dokument speichern und Inhalt Speichern | button.text Dokument speichern | TEXT Speichern |

| | | |
|---|---|---|
| button name=action value=save mit Inhalt Speichern | button.text Speichern | TEXT Speichern; EXTRACT attribute value für den Formularwert |

`value` eines button kann ein übermittelter Formularwert sein, ohne seine sichtbare Beschriftung zu bilden. VALUE-Prüfungen im aktuellen AGATE-Code verwenden Playwright hasValue und sind für input-Werte vorgesehen; sie sind keine allgemeine Attributprüfung beliebiger button-Elemente. Für ein button-Attribut stattdessen EXTRACT mit attribute verwenden.

### 3. Vollständiger vereinfachter YAML-Vertrag

#### Felder des Schritts

| Feld | Typ | Pflicht / Standard | Bedeutung |
|---|---|---|---|
| type | String | Pflicht | GUI. |
| op | String | Explizit angeben | EXEC für Aktion, ASSERT für Prüfung. |
| action | String | Pflicht bei dieser Form | Einer der weiter unten dokumentierten Werte. |
| id | String | Optional, empfohlen | Testschritt-ID für die Diagnose; keine HTML-ID. |
| page | String | Optional, empfohlen | Beschreibung im Log; wählt keinen Tab. |
| button | Objekt | Alternative zu target | Vereinfachte Buttonauswahl. |
| target | Objekt | Alternative zu button | Technische Auswahl mit genau einer |

| | | | |
|---|---|---|---|
| | | | Locator-Strategie. |
| timeout | Integer | Optional | Millisekunden; Schrittwert, sonst TestCase-Wert, sonst 30000. |
| expected | String/Integer | Abhängig von ASSERT | Erwarteter Inhalt oder Anzahl. |

**Alle erlaubten Felder innerhalb button**

| Feld | Typ | Pflicht / Standard | Erklärung |
|---|---|---|---|
| text | Nicht leerer String | Genau eines von text oder id | Zugänglicher Buttonname. |
| id | Nicht leerer String | Genau eines von text oder id | HTML-ID eines nativen Buttons oder input button/submit/reset. |
| exact | Boolean | Optional bei text; Standard true | true: vollständiger normalisierter Name, case-sensitive. false: Teilstring, case-insensitive. |
| index | Nicht negativer Integer | Optional | Nultbasierte Position innerhalb der Treffer. |

Nicht gleichzeitig text und id setzen. Bei id muss exact fehlen. Nicht button mit target, selector oder einer zweiten vereinfachten Elementform kombinieren. Boolean und Integer nicht als Strings schreiben: exact true und index 0 ohne Anführungszeichen.

`button.id` wird im geprüften Code in diesen CSS-Ausdruck übersetzt:

```
:is(button,input[type=button],input[type=submit],input[type=reset])
[id="BEISPIEL-ID"]
```

Dabei wird die konkrete ID korrekt als CSS-String escaped. Der hier dargestellte Platzhalter ist keine echte ID. Ein div role=button mit ID wird nicht durch button.id erfasst; dafür target.role oder target.css verwenden. Ebenso zählt input type=image nicht zu dieser ID-Tagliste, auch wenn ein entsprechender Rollen-Locator passen kann.

#### exact anhand der vorhandenen L02-Seite

Die Lernseite enthält `(O)K` als input-Beschriftung und `(O)K continue` als button-Text.

| Auswahl | Ergebnis |
|---|---|
| text `(O)K`, exact true | Nur der input-Button; das äußere Leerzeichen wird normalisiert. |
| text `(O)K`, exact weggelassen | Dasselbe: AGATE-Standard true. |
| text `(O)K`, exact false | Beide Buttons; CLICK ist mehrdeutig. COUNT kann 2 prüfen. |
| text `(o)k`, exact true | Abweichende Groß-/Kleinschreibung passt nicht. |
| text `(o)k`, exact false | Teilstring ohne Beachtung der Groß-/Kleinschreibung. |
| text `(O)K*`, exact true | Sucht eine echte Sternchen-Beschriftung; kein Glob. |

Auch exact true garantiert keine Einzigartigkeit: Zwei gleichnamige Buttons bleiben zwei Treffer. index ist kein Ersatz für eine fachlich begründete Eingrenzung.

#### Technische Alternativen

| target-Feld | Verwendung |
|---|---|
| role: button plus name und exact | Zugängliche Rolle und Name; auch für korrekt implementierte ARIA-Buttons. |
| css | Stabile ID oder andere belegte DOM-Bedingung. |

| | |
|---|---|
| testId | data-testid als vereinbarter Testvertrag. |
| text plus exact | Allgemeine Textauswahl; nicht automatisch auf Buttonrolle beschränkt. |
| scope | Suchbereich als weiteres target-Objekt, etwa ein bestimmter Dialog. |
| frames | Liste von Iframe-CSS-Selektoren von außen nach innen. |
| nth | Nultbasierter Index der technischen Locator-Treffer. |

Genau eine Strategie im target wählen; name ergänzt role. scope, frames und nth sind zusätzliche Einschränkungen, keine alleinigen Strategien. `button.css`, `button.label`, `button.value`, `button.regex`, `button.textGlob`, `button.scope` und `button.frames` sind keine erlaubten Felder der vereinfachten Form.

**4. Vollständige EXEC-Aktionsreferenz für Buttons**

| action bei op EXEC | Verhalten | Zusätzliche Parameter | Voraussetzung |
|---|---|---|---|
| CLICK | Button anklicken; Ausführbarkeit automatisch abwarten. | Keine weiteren Pflichtfelder. | Eindeutiger ausführbarer Button. |
| HOVER | Mauszeiger auf Button bewegen. | Keine weiteren Pflichtfelder. | Geeignet für Tooltip oder Hover-Verhalten. |
| PRESS | Taste/Tastenkombination an das Element senden. | value erforderlich. | Enter oder Space kann native Buttons aktivieren; Verhalten prüfen. |
| WAIT | Auf den Locator-Zustand warten. | state optional: attached, detached, | WAIT auf visible prüft nicht |

| | | | |
|---|---|---|---|
| | | visible, hidden; Standard visible. | automatisch enabled. |
| EXTRACT | Text oder Attribut lesen und speichern. | attribute oder source optional; buffer optional, alternativ response. | attribute z. B. type oder value; source value für input; sonst innerText. Standardbuffer extracted. |
| DOWNLOAD | Klicken, Downloadereignis abwarten und Datei speichern. | value optional als Dateiname; buffer optional, Standard download. | Die Buttonaktion muss tatsächlich einen Browserdownload erzeugen. |
| DOWNLOAD_PDF | Originalen PDF-Inhalt eines passenden POST erfassen und speichern. | outputFile und responseUrlGlob erforderlich; buffer optional, Standard download; ignoreHttpsErrors optional. | Spezielle POST-PDF-Verarbeitung, keine universelle HTML-Druck- oder GET-Funktion. |

timeout ist für alle Aktionen optional und folgt der beschriebenen Priorität. Ist ein EXTRACT-Attribut nicht vorhanden, wird ein Fehler ausgelöst. Ein extrahierter Wert ist noch keine Inhaltsprüfung.

FILL/INPUT, SELECT, CHECK, UNCHECK und UPLOAD sind zwar allgemeine GUI-Funktionen, aber keine sinnvollen Aktionen für gewöhnliche Buttons. OPEN, BACK, SCREENSHOT, CLOSE und Tabverwaltung betreffen Seite oder Session. Sie werden auf der Hauptseite bzw. in ihren eigenen Kapiteln beschrieben. Ein Button mit aria-pressed wird nicht durch CHECK zu einer Checkbox.

### 5. Vollständige ASSERT-Referenz

#### Zustände, Treffer und Text

| action bei op ASSERT | Bedeutung | expected |
|---|---|---|
| EXISTS | Element ist im DOM vorhanden; muss nicht | Nicht erforderlich. |

| | | |
|---|---|---|
| | sichtbar sein. | |
| NOT_EXISTS | Keine Locator-Treffer. | Nicht erforderlich. |
| VISIBLE | Element ist sichtbar. | Nicht erforderlich. |
| HIDDEN | Element ist unsichtbar oder nicht vorhanden. | Nicht erforderlich. |
| ENABLED | Element ist aktiviert nach Playwright-Semantik. | Nicht erforderlich. |
| DISABLED | Element ist deaktiviert nach Playwright-Semantik. | Nicht erforderlich. |
| COUNT | Exakte Trefferanzahl. | Integer erforderlich. |
| TEXT | Vollständiger normalisierter innerText. | String erforderlich. |
| TEXT_EQUALS | Alias von TEXT; derselbe Codezweig. | String erforderlich. |
| TEXT_CONTAINS | Normalisierter Text enthält einen literalen Teilstring. | String erforderlich; kein Joker. |
| TEXT_GLOB | Vollständiger Text mit normalisierten Leerzeichen/NBSP und Joker *. | String erforderlich; ? ist im AGATE-Vertrag literal. |
| TEXT_MATCHES | Regex gegen innerText, DOTALL. | Regex-String erforderlich; ^ und $ für vollständige Grenzen. |

TEXT-Varianten eignen sich für Buttons mit Elementtext. Bei input button/submit/reset stattdessen die folgenden VALUE-Varianten verwenden. Die Sonderbehandlung von Input-value beim Finden eines Namens bedeutet nicht, dass innerText denselben Wert liefert.

#### Zusätzliche Wertprüfungen für input-Buttons

| action bei op ASSERT | Bedeutung | expected |
|---|---|---|

| | | |
|---|---|---|
| VALUE | Exakter Eingabewert, inklusive bedeutender Leerzeichen. | String erforderlich. |
| VALUE_GLOB | Eingabewert mit Joker *, ohne die Text-Whitespace-Normalisierung. | String erforderlich; ? literal. |
| VALUE_MATCHES | Regex gegen den Eingabewert, DOTALL. | Regex-String erforderlich. |

Bei L02-ok lautet value `(O)K ` mit abschließendem Leerzeichen. Deshalb prüft VALUE diesen Wert, während button.text `(O)K` als Locator ausreicht. `VALUE_GLOB: '(O)K*'` kann das nachfolgende Leerzeichen einschließen. Bei einem normalen button-Element ist dies keine Alternative zu TEXT.

CHECKED und UNCHECKED sind für Checkbox-/Radio-Semantik implementiert, nicht für gewöhnliche Buttons. Eine reine CSS-Klasse disabled ist nicht gleichwertig mit disabled oder wirksamer aria-disabled-Semantik. Sichtbar und aktiviert sind unterschiedliche Zustände. ENABLED beweist keine erfolgreiche fachliche Verarbeitung.

### 6. Timeout, Screenshots und Sections

Schritt-timeout hat Vorrang vor dem TestCase-timeout; fehlen beide, gilt 30000 ms im GuiStepConfig-Code. Bei der Demo steht auf TestCase-Ebene 10000 ms. Die Aktionen müssen die gesamte Zeit nicht ausschöpfen. Eine feste Lernpause ist ein eigener type WAIT mit value 5000.

Bei Fehlern im GUI-Ausführungsblock versucht die Engine automatisch einen Screenshot und nennt ihn unter Evidence. Locator, Quell-YAML und Quellschritt gemeinsam prüfen. Ein Screenshot vor Browserstart oder bei bereits geschlossener Seite ist nicht garantiert.

Section ist eine informative Laufzeitmarkierung mit der Section-Korrektur. Sie kann auf Deutsch erklären, was als Nächstes getestet wird; sie ist kein eigener ausführbarer Statistikschritt. Die Loglabels der Engine bleiben englisch.

### 7. Ausführbarer Einstieg mit der vorhandenen L02-Fixture

Dieses Beispiel passt zur vorhandenen Seite `02-buttons.html`. Es benötigt keine neuen HTML-Elemente. Die Assertions prüfen den jeweiligen Zustand; die 5-Sekunden-Pause dient nur der Beobachtung.

```
name: Buttons – Locator und Beschriftung
timeout: 10000
steps:
  - section: Lernseite öffnen
```

```
    description: Native Buttons, Input-Buttons und verschachtelte Beschriftungen zeigen.
  - type: GUI
    op: EXEC
    action: OPEN
    url: 'http://127.0.0.1:8767/02-buttons.html'

  - section: Exakter Buttonname
    description: Das abschließende Leerzeichen im Input-value beeinflusst die Namensauswahl nicht.
  - type: GUI
    op: ASSERT
    action: VISIBLE
    button:
      text: '(O)K'
      exact: true

  # exact=true ist optional und wird hier nicht angegeben.
  - type: GUI
    op: ASSERT
    action: VISIBLE
    button:
      text: '(O)K'

  - section: Teilstring und Anzahl
    description: Die nicht exakte Auswahl findet zwei Buttons; kein mehrdeutiger CLICK.
  - type: GUI
    op: ASSERT
    action: COUNT
    button:
      text: '(O)K'
      exact: false
    expected: 2

  - section: Input-value prüfen
    description: VALUE vergleicht die echte Input-Beschriftung mit abschließendem Leerzeichen.
  - type: GUI
    op: ASSERT
    action: VALUE
    button:
      id: L02-ok
    expected: '(O)K '
  - type: GUI
    op: ASSERT
    action: VALUE_GLOB
    button:
      id: L02-ok
    expected: '(O)K*'
  - type: GUI
    op: ASSERT
    action: VALUE_MATCHES
    button:
      id: L02-ok
    expected: '^\(O\)K\s*$'

  - section: Klick und Ergebnis
    description: Den Button aktivieren und die sichtbare Wirkung prüfen.
  - type: GUI
    op: EXEC
    action: CLICK
    button:
      text: '(O)K'
      exact: true
  - type: GUI
    op: ASSERT
    action: TEXT
    target:
      css: '#L02-result'
    expected: OK clicked
```

```yaml
  - type: WAIT
    value: '5000'

  - section: Verschachtelter Text
    description: Ein span im Button erfordert keinen separaten span-Locator.
  - type: GUI
    op: EXEC
    action: CLICK
    button:
      text: Nested action
      exact: true
  - type: GUI
    op: ASSERT
    action: TEXT
    target:
      css: '#L02-result'
    expected: Nested clicked

  # Alternative Auswahl desselben Buttons über die HTML-ID.
  - type: GUI
    op: ASSERT
    action: VISIBLE
    button:
      id: L02-nested

  # Alternative Auswahl über den Testvertrag.
  - type: GUI
    op: ASSERT
    action: VISIBLE
    target:
      testId: nested-button

  # Alternative Auswahl über Rolle und zugänglichen Namen.
  - type: GUI
    op: ASSERT
    action: VISIBLE
    target:
      role: button
      name: Nested action
      exact: true

  - section: Tastaturaktion
    description: Enter aktiviert den nativen Button; danach den Ergebnistext prüfen.
  - type: GUI
    op: EXEC
    action: PRESS
    button:
      id: L02-continue
    value: Enter
  - type: GUI
    op: ASSERT
    action: TEXT
    target:
      css: '#L02-result'
    expected: Continue clicked
```

Dieses Einstiegsbeispiel ist keine Behauptung, dass bereits jede dokumentierte Zustands- oder Downloadvariante auf L02 existiert. Die zusätzlichen Beispiele im folgenden Abschnitt benötigen ihre ausdrücklich genannten Fixtures.

### 8. Doppelte Namen mit scope eingrenzen

Zusätzliches HTML-Beispiel, nicht Teil der bisherigen L02-Fixture:

```html
<section id="editor"><button type="button">Speichern</button></section>
```

```
<section id="settings"><button type="button">Speichern</button>
</section>
```

```
- type: GUI
  op: EXEC
  action: CLICK
  target:
    role: button
    name: Speichern
    exact: true
    scope:
      css: '#editor'
```

Die zwei Repeat-Buttons der aktuellen L02-Seite zeigen bereits die Mehrdeutigkeit. COUNT mit button.text Repeat kann 2 prüfen. Ein index 0 wählt den ersten Treffer, begründet aber nicht, warum dieser fachlich der richtige ist.

### 9. Einzelbeispiele für jede Aktion und Prüfung

Die folgenden Fragmente erläutern alle dokumentierten Werte. Sie sind keine gemeinsame positive Suite. Insbesondere benötigen HOVER mit Wirkung, Zustandswechsel und Downloads passende zusätzliche HTML-Fixtures. Namen mit dem Präfix Zusatzfixture werden nicht als vorhandene Demo-IDs dargestellt.

#### EXEC / CLICK

```
# Den vorhandenen Input-Button anklicken.
- type: GUI
  op: EXEC
  action: CLICK
  button:
    id: L02-ok
```

#### EXEC / HOVER

```
# Den vorhandenen Button mit der Maus ansteuern. Eine Tooltip-Wirkung
erfordert eine zusätzliche Fixture.
- type: GUI
  op: EXEC
  action: HOVER
  button:
    id: L02-nested
```

#### EXEC / PRESS

```
# Den vorhandenen nativen Button mit Enter aktivieren.
- type: GUI
  op: EXEC
  action: PRESS
  button:
    id: L02-continue
  value: Enter
```

#### EXEC / WAIT

```
# Auf Sichtbarkeit warten, nicht pauschal 2500 ms schlafen.
- type: GUI
  op: EXEC
  action: WAIT
  button:
    id: L02-nested
  state: visible
  timeout: 2500
```

### EXEC / EXTRACT

```yaml
# Die Input-Beschriftung als Attribut lesen. Danach den Buffer verwenden oder prüfen.
- type: GUI
  op: EXEC
  action: EXTRACT
  button:
    id: L02-ok
  attribute: value
  buffer: DemoButtonValue
```

### EXEC / DOWNLOAD

```yaml
# Zusatzfixture erforderlich: Dieser Button muss ein echtes Downloadereignis auslösen.
- type: GUI
  op: EXEC
  action: DOWNLOAD
  button:
    id: Zusatzfixture-download-button
  value: button-download.txt
  buffer: DemoButtonDownload
```

### EXEC / DOWNLOAD_PDF

```yaml
# Vorher die vorhandene Seite 11-files.html öffnen und den neutralen Demo-Service starten. Der Submit-Button sendet den PDF-POST.
- type: GUI
  op: EXEC
  action: DOWNLOAD_PDF
  button:
    id: L11-pdf
  outputFile: output/learningdemo/button-original.pdf
  responseUrlGlob: '**/demo-report.pdf*'
  buffer: DemoButtonPDF
  timeout: 10000
```

### ASSERT / EXISTS

```yaml
# Den vorhandenen nativen Button prüfen.
- type: GUI
  op: ASSERT
  action: EXISTS
  button:
    id: L02-nested
```

### ASSERT / NOT_EXISTS

```yaml
# Die Abwesenheit erfüllt NOT_EXISTS und auch HIDDEN. EXISTS würde fehlschlagen.
- type: GUI
  op: ASSERT
  action: NOT_EXISTS
  button:
    id: nicht-vorhandener-button
```

### ASSERT / VISIBLE

```yaml
# Den vorhandenen nativen Button prüfen.
- type: GUI
  op: ASSERT
  action: VISIBLE
  button:
    id: L02-nested
```

### ASSERT / HIDDEN

```yaml
# Die Abwesenheit erfüllt NOT_EXISTS und auch HIDDEN. EXISTS würde fehlschlagen.
- type: GUI
  op: ASSERT
  action: HIDDEN
  button:
    id: nicht-vorhandener-button
```

### ASSERT / ENABLED

```yaml
# Den vorhandenen nativen Button prüfen.
- type: GUI
  op: ASSERT
  action: ENABLED
  button:
    id: L02-nested
```

### ASSERT / DISABLED

```yaml
# Zusatzfixture erforderlich, beispielsweise button disabled mit dieser ID.
- type: GUI
  op: ASSERT
  action: DISABLED
  button:
    id: Zusatzfixture-disabled-button
```

### ASSERT / COUNT

```yaml
# Mehrfachtreffer bewusst über COUNT prüfen.
- type: GUI
  op: ASSERT
  action: COUNT
  button:
    text: Repeat
    exact: true
  expected: 2
```

### ASSERT / TEXT

```yaml
# Den vorhandenen nativen Button prüfen.
- type: GUI
  op: ASSERT
  action: TEXT
  button:
    id: L02-nested
  expected: Nested action
```

### ASSERT / TEXT_EQUALS

```yaml
# Den vorhandenen nativen Button prüfen.
- type: GUI
  op: ASSERT
  action: TEXT_EQUALS
  button:
    id: L02-nested
  expected: Nested action
```

### ASSERT / TEXT_CONTAINS

```yaml
# Den vorhandenen nativen Button prüfen.
- type: GUI
  op: ASSERT
  action: TEXT_CONTAINS
  button:
    id: L02-nested
  expected: action
```

ASSERT / TEXT_GLOB

```yaml
# Den vorhandenen nativen Button prüfen.
- type: GUI
  op: ASSERT
  action: TEXT_GLOB
  button:
    id: L02-nested
  expected: Nested*
```

ASSERT / TEXT_MATCHES

```yaml
# Den vorhandenen nativen Button prüfen.
- type: GUI
  op: ASSERT
  action: TEXT_MATCHES
  button:
    id: L02-nested
  expected: ^Nested action$
```

ASSERT / VALUE

```yaml
# Input-Button: den tatsächlichen value prüfen, nicht innerText.
- type: GUI
  op: ASSERT
  action: VALUE
  button:
    id: L02-ok
  expected: '(O)K '
```

ASSERT / VALUE_GLOB

```yaml
# Input-Button: den tatsächlichen value prüfen, nicht innerText.
- type: GUI
  op: ASSERT
  action: VALUE_GLOB
  button:
    id: L02-ok
  expected: (O)K*
```

ASSERT / VALUE_MATCHES

```yaml
# Input-Button: den tatsächlichen value prüfen, nicht innerText.
- type: GUI
  op: ASSERT
  action: VALUE_MATCHES
  button:
    id: L02-ok
  expected: ^\(O\)K\s*$
```

### 10. Zustandsfixtures für die nächste Demo-Erweiterung

Folgende zusätzliche HTML-Elemente würden die bisher fehlenden Zustandsbeispiele ermöglichen. Dieser Quelltext ist ein Vorschlag zum Einbau, kein bereits ausgeführtes Demo-Update:

```html
<button id="Zusatzfixture-disabled-button" type="button"
disabled>Später verfügbar</button>
<button id="Zusatzfixture-hidden-button" type="button"
hidden>Unsichtbare Aktion</button>
<button id="Zusatzfixture-toggle" type="button" aria-
pressed="false">Ansicht wechseln</button>
```

Der erste Button ist vorhanden und deaktiviert; der zweite vorhanden und versteckt. Ein ARIA-Toggle-Button besitzt einen pressed-Zustand und ist keine Checkbox. Der aktuelle vereinfachte Buttonvertrag besitzt kein Feld pressed; auch ASSERT CHECKED ist dafür nicht die richtige Prüfung. Ein aria-pressed-Attribut kann mit EXTRACT gelesen werden, aber dieser Lesevorgang allein prüft nicht den erwarteten Wert.

**11. Fehleranalyse**

| Problem | Wahrscheinliche Prüfung | Konkrete Korrektur |
|---|---|---|
| CLICK findet mehrere Buttons | COUNT und gleiche Namen | Scope oder stabile ID; nicht blind index 0. |
| exact=true findet nichts | Zugänglichen Namen, ARIA und Groß-/Kleinschreibung prüfen | Namen korrigieren; nicht generell exact=false setzen. |
| input value hat ein Leerzeichen | CSS prüft den Rohwert | Buttonname verwenden oder absichtlich korrekten CSS-Wert angeben. |
| TEXT liest beim input keinen Beschriftungstext | HTML-Elementtyp | VALUE-Prüfung statt TEXT. |
| VALUE auf normalem button scheitert | inputValue-Semantik | TEXT für Beschriftung, EXTRACT für ein Attribut. |
| Button sichtbar, CLICK wartet | disabled, Overlay, Layout und aktiven Frame prüfen | Zustandsänderung abwarten; Locator-/Seitenfehler nicht durch längere Pausen verstecken. |
| Formular wird unerwartet abgesendet | type und Formularkontext | In der Anwendung richtigen Buttontyp setzen; im Test tatsächliches Verhalten berücksichtigen. |

| | | |
|---|---|---|
| DOWNLOAD wartet ohne Ereignis | Tatsächliche Reaktion auf CLICK | Navigationsantwort nicht als Download behandeln. |
| DOWNLOAD_PDF bekommt 404 | POST-Route und laufende Demo-Version | Neuen neutralen Service starten; /demo-report.pdf prüfen. |
| PDF-Tab bleibt offen | Stand der PDF-Lebensdauer-Korrektur | GuiPdfDownload-Korrektur 1.0.65 verwenden und reales Browserverhalten prüfen. |
| Ungültige button-Felder | Schema prüfen | Nur text/id/exact/index; für scope/frames/CSS target verwenden. |

Timeout allein beweist keine bestimmte Fehlerursache. Quell-YAML, Schritt, aktueller URL, Locator und Screenshot zusammen betrachten. Ein Klickerfolg ist nur der Nachweis der ausgeführten GUI-Aktion; die fachliche Wirkung benötigt eine zusätzliche Prüfung.

### 12. AGATE Best Practice

Native Buttons und klare Namen bevorzugen. Bei übersetzten Beschriftungen stabile IDs oder testIds vereinbaren. Den tatsächlichen HTML-Typ prüfen, bevor TEXT oder VALUE gewählt wird. Nach CLICK oder PRESS Ergebniszustand statt nur Klickstatus prüfen. Mehrdeutige Namen mit scope eingrenzen. Für Lernpausen normale WAIT-Schritte verwenden, für Synchronisation Assertions oder GUI WAIT.